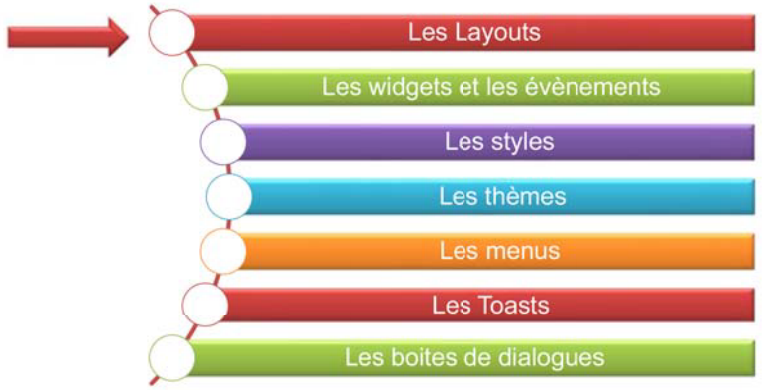
PLAN

- 1. Introduction programmation mobile (Android)
- 2. Environnement de Développement
- 3. Cycle de Vie d'une Activité
- **4. Interfaces graphiques**
- 5. Les Intents
- 6. Persistance des données
- 7. Capteurs sous Android
- 8. Programmation Concurrente

4. Interfaces graphiques



Éléments Graphiques de Base: Unités de mesure

- **pixel (px) : correspond à un pixel de l'écran**
- **pouce (in) : unité de longueur, correspondant à 2,54 cm. Basé sur la taille physique de l'écran**
- **millimètre (mm) : basé sur la taille physique de l'écran**
- **point (pt) : 1/72 in**
- **pixel à densité indépendante (dp ou dip) : une unité relative se basant sur une taille physique de l'écran de 160 dpi.**
  - Avec cette unité, 1 dp est égal à 1 pixel sur un écran de 160 pixels.
  - Si la taille de l'écran est différente de 160 pixels, les vues s'adapteront selon le ratio entre la taille en pixels de l'écran de l'utilisateur et la référence des 160 pixels d'un pouce
  - L'avantage de cette unité est qu'elle permet de gérer plus facilement une grande variété d'écrans
- **pixel à taille indépendante (sp) : fonctionne de la même manière que les pixels à densité indépendante à l'exception qu'ils sont aussi fonction de la taille de polices spécifiée par l'utilisateur. Il est recommandé d'utiliser cette unité lorsque vous spécifiez les tailles des polices.**

Éléments Graphiques de Base: Unités de mesure



Éléments Graphiques de Base: Les Composants View et ViewGroup

- **View**:
  - Classe de base pour la création d'une interface graphique en Andrcid
  - **Classe mère** de tous les widgets (éléments graphiques)
  - Une vue peut être ajoutée à une application :
    - Soit statiquement, dans le fichier de description de l'interface (Layout XML)
    - Soit dynamiquement, dans le code Java de l'application
- **ViewGroup**:
  - Classe de base de tous les **layouts** (Gestionnaire de dispositions)
  - **Layouts**: Conteneurs invisibles regroupant plusieurs View ou ViewGroup suivant une **prédisposition** des objets graphiques.
  - Permet de définir l'emplacement de ses fils dans l'écran

Éléments Graphiques de Base: Les Composants View et ViewGroup



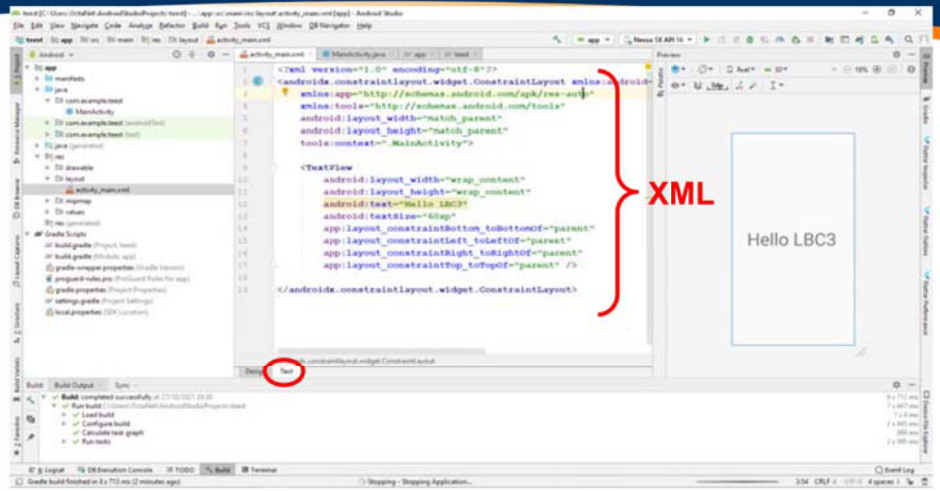
Éléments Graphiques de Base: Les Interfaces graphiques

- **Interfaces graphiques : Les interfaces permettent de dessiner la vue tel qu'elle doit s'afficher à l'utilisateur. Il existe deux possibilités de développer des interfaces :**
  - **Code java**
  - **Fichier XML**
    - **Android recommande l'utilisation des fichiers XML pour définir les interfaces : séparer les vues de la couche métiers.**
- **Une interface XML peut être associée à une activité (vue) grâce à la méthode *setContentView*(identifiant_layout); Exemple : setContentView(R.layout.*login*);**
- **Widget(View) : Éléments graphiques constituant la vue.**
  - **Widgets simples : boutons, champs de saisie, radios, listes, ...**
  - **Widgets complexes : datepicker, horloge, barre de progression ...**
    - **Définition en XML, et implémentation du comportement en JAVA.**

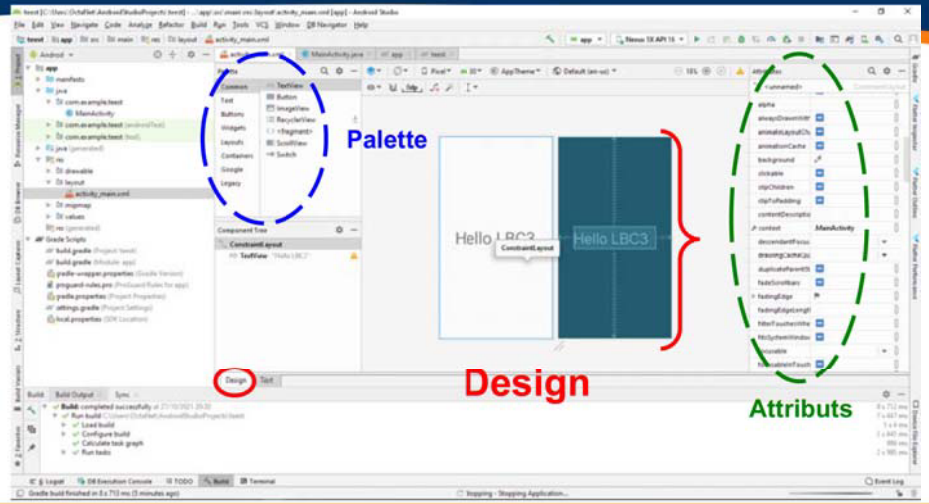
Éléments Graphiques de Base: Les Interfaces graphiques

- **Placés dans le répertoire res/layout de l'arborescence du projet**
- **Possibilité de visualiser le contenu du fichier:**
  - **Sous forme de code XML (onglet Text)**
  - **Sous forme graphique (onglet Design) : mise à disposition d'une palette de composants graphiques à glisser-déplacer vers l'emplacement désiré**
- **Chaque composant graphique est représenté par un élément XML, dont les attributs décrivent l'aspect ou le comportement.**

Éléments Graphiques de Base: Les Interfaces graphiques

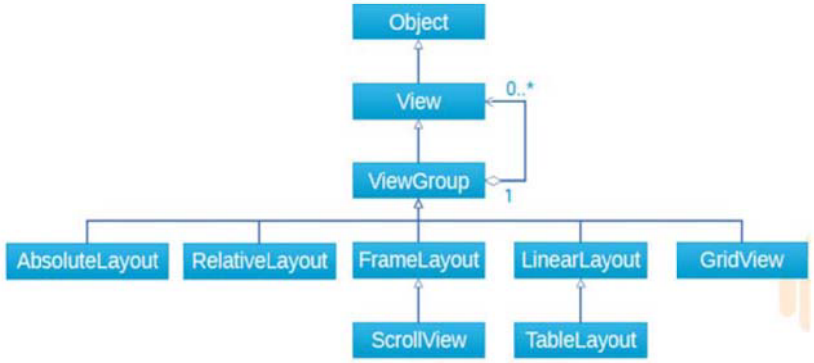


Éléments Graphiques de Base: Les Interfaces graphiques



Éléments Graphiques de Base: Layouts: disposition

- Des conteneurs capables de placer les composants qu'ils contiennent de façon automatique



Éléments Graphiques de Base: Layouts: vue d'ensemble

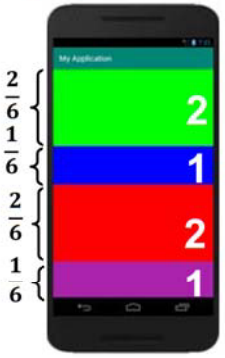
- **AbsoluteLayout (*deprecated*) : position absolue (x et y)**
- **LinearLayout : permet d'aligner de gauche à droite ou de haut en bas les éléments qui y seront intégrés. (orientation = horizontal / vertical)**
- **RelativeLayout : les composants sont positionnés les uns par rapport aux autres, le premier composant servant de référence aux autres**
- **TableLayout : crée un tableau à deux dimensions organisées par colonne et par rangée**
- **GridLayout : grille flexible, liste d'éléments continus (de gauche à droite)**
- **FrameLayout : Ce layout est principalement utilisé pour l'affichage d'un élément (par exemple, un cadre dans lequel on veut charger des images)**
- **ScrollView / HorizontalScrollView : permet de défiler un contenu trop grand pour tenir sur l'écran**

Éléments Graphiques de Base: Attributs Layouts Communs

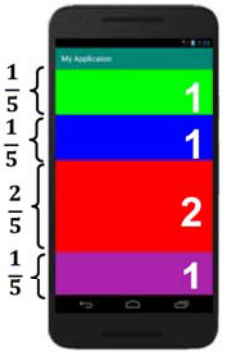
- **Identifiant : android:id="@+id/mon_ident"**
- **android:layout_width et android:layout_height:**
  - **="mach_parent"**: l'élément remplit tout l'élément parent
  - **="wrap_content"**: prend la place nécessaire à l'affichage
- **android:margin:** définit la **taille des marges** sur les quatre côtés de l'élément. C'est une propriété raccourcie qui permet de manipuler les autres propriétés de marges : margin-top, margin-right, margin-bottom et margin-left
- **android:background: "@color/blue/#0000FF"**
- **android:visibility : visible / invisible**
- **android:layout_gravity :** est la gravité **extérieure** de la vue. Spécifie la direction dans laquelle la vue doit toucher la bordure de son parent. **left | center_horizontal | top | bottom | right**
- **android:gravity:** est la gravité **intérieure** de cette vue. Spécifie dans quelle direction son contenu doit s'aligner. **left | center_horizontal | top | bottom | right**

Éléments Graphiques de Base: Attributs Layouts

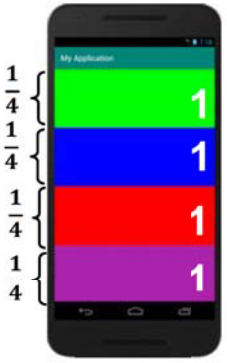
- **Layout_weight : change la taille de la zone occupée par chaque composant (distribution)**



La somme des weight : 6



La somme des weight : 5



La somme des weight : 4

Éléments Graphiques de Base: AbsoluteLayout

AbsoluteLayout

- **Placement des composants par positionnement**
- **Propriétés des composants :**
  - **android:layout_x**
  - **android.layout_y**

```
<AbsoluteLayout
    android:id="@+id/absoluteLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button"
        android:layout_x="80dp"
        android:layout_y="200dp"/>

</AbsoluteLayout>
```



Éléments Graphiques de Base: LinearLayout

LinearLayout

- **Position en colonnes ou en lignes**
- **Deux orientations possibles suivant la propriété android:layout_orientation**
  - **vertical : positionne les vues les unes au-dessus des autres**
  - **horizontal : positionne les vues les unes à côté des autres**
- **Les conteneurs ou les widgets appartenant à ce Layout sont soit alignés en ligne horizontale ou verticale.**
- **Poids :**
  - **android:layout_weight : détermine le poids du composant par rapport aux autres**
  - **Plus le poids est important, plus le composant prend de la place**
- **Gravity :**
  - **Gestion de l'espace vide : gère l'emplacement du composant dans la case**
  - **android:gravity organise le contenu à l' intérieur de la vue.**
  - **android:layout_gravity organise la position de la vue en dehors d'elle-même**

Éléments Graphiques de Base: LinearLayout : exemple

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center|top"
    android:orientation="vertical">
    <Button
        android:id="@+id/button3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button 1" />
    <Button
        android:id="@+id/but2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button 2"/>
    <Button
        android:id="@+id/button4"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Button 3" />
    <Button
        android:id="@+id/but5"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Button 4"/>
</LinearLayout>
```



Éléments Graphiques de Base: RelativeLayout

- **Le principe du RelativeLayout est de placer les éléments selon d'autres éléments du conteneur.**
- **Positionnement relatif au conteneur :**
  - android:layout_alignParentTop (true / false) : Cette option permet de préciser si le haut de l'élément doit être aligné avec celui de son conteneur.
- **De même pour :**
  - android:layout_alignParentBottom,
  - android:layout_alignParentLeft
  - android:layout_alignParentRight.

Éléments Graphiques de Base: RelativeLayout

- **Position relative aux autres éléments : Afin de pouvoir référencer le positionnement d'un élément par rapport à un autre, vous disposez d'un moyen simple et efficace, il s'agit des identificateurs id**
  - **A la déclaration d'un élément : android:id= "@+id/idElem"**
  - **A l'utilisation : @id/idElem**
  - `android:layout_above` : Indique que l'élément sera placé au-dessus de celui indiqué par son id.
  - `android:layout_below` : Indique que l'élément sera placé en dessous de celui indiqué par son id.
  - `android:layout_toLeftOf` : Indique que l'élément sera placé à gauche de celui indiqué par son id.
  - `android:layout_toRightOf` : Indique que l'élément sera placé à droite de celui indiqué par son id.
  - `android:layout_alignTop` : Indique que le haut de l'élément est aligné avec le haut de l'élément indiqué.

Éléments Graphiques de Base: RelativeLayout

- `android:layout_alignBottom` : Indique que le bas de notre élément est aligné avec le bas de l'élément indiqué.
- `android:layout_alignLeft` : Indique que le côté gauche de notre élément est aligné avec le côté gauche de l'élément indiqué.
- `android:layout_alignRight` : Indique que le côté droit de notre élément est aligné avec le côté droit de l'élément indiqué.
- `android:layout_alignBaseLine` : Indique que les lignes de base des 2 éléments sont alignées.

- Jxufhjr jsy%f{jh%gjx%ntr utxfsyx%wjqfyhx%
  - `layout_marginTop`,
  - `layout_marginLeft`,
  - `layout_marginBottom`,
  - `layout_marginRight`

Éléments Graphiques de Base: RelativeLayout

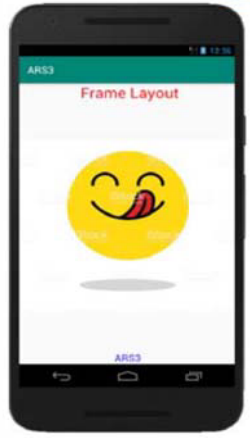
```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/bt1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="button 1"
        android:layout_alignParentTop="true"
        android:layout_alignParentRight="true"/>
    <Button
        android:id="@+id/bt2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="bt2"
        android:layout_alignParentRight="true"
        android:layout_below="@id/bt1"/>
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="bt3"
        android:layout_toLeftOf="@id/bt2"
        android:layout_below="@id/bt2"/>
</RelativeLayout>
```



Éléments Graphiques de Base: FrameLayout

- Le FrameLayout est utilisé généralement pour superposer des views ensembles.

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        app:srcCompat="@drawable/ist" />
    <TextView
        android:id="@+id/textView3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center|top"
        android:text=" Frame Layout"
        android:textSize="30dp"
        android:textColor="#FF0000"/>
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center|bottom"
        android:text="ARS3"
        android:textSize="20dp"
        android:textColor="#0000FF"/>
</FrameLayout>
```



Éléments Graphiques de Base: TableLayout

- Arranger les éléments dans des linges et colonnes sous forme d'un tableau.
  - TableRow : conteneur de ligne
  - layout_span : un widget pourra occuper plusieurs cellules
  - layout_column : le numéro de colonne où sera placé le widget (0+)
- La construction est réalisée ligne par ligne

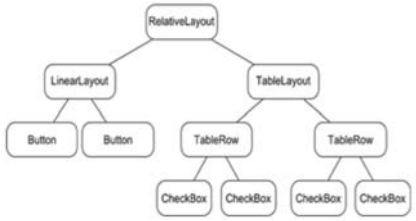
```
<TableLayout>
    <TableRow>      </TableRow>
    <TableRow>      </TableRow>
    <TableRow>      </TableRow>
</TableLayout>
```

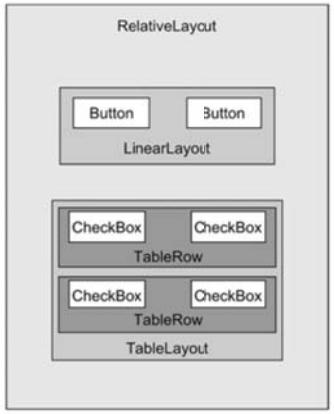

Éléments Graphiques de Base: TableLayout

```
<TableLayout
    android:shrinkColumns="2"
    android:stretchColumns="2"
    android:layout_height="match_parent"
    android:layout_width="match_parent" >
    <TableRow
        android:id="@+id/tableRow1"
        android:paddingTop="50dp" >
        <Button
            android:id="@+id/bt1"
            android:text="button 1"/>
        <Button
            android:id="@+id/bt2"
            android:text="button 2"/>
        <Button
            android:id="@+id/bt3"
            android:text="button 3"/>
    </TableRow>
    <TableRow android:id="@+id/tableRow2" >
        <Button android:id="@+id/button1" />
        <Button
            android:id="@+id/button2"
            android:layout_span="2" />
    </TableRow>
</TableLayout>
```



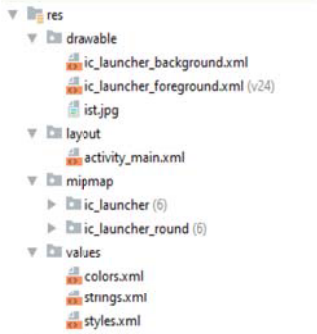
Éléments Graphiques de Base: Combinaison Layout





Éléments Graphiques de Base: Les Ressources

- Les ressources sont dans le répertoire res.
- Elles sont organisées dans des répertoires qui contiennent :
  - Des images,
  - Des chaines de caractères,
  - Des couleurs,
  - Fichiers XML.



Éléments Graphiques de Base: Les Ressources : Classe R

- Le framework Android génère et maintient à jour automatiquement une classe statique R.java contenant les identifiants (valeur entière) des ressources.
- La classe R se trouve sous le dossier app/build/generated mais qui n'apparait pas dans l'arborescence de l'IDE.
- A chaque fois qu'on rajoute une ressource à l'application, un identifiant est généré automatiquement dans cette classe pour permettre par la suite de pouvoir le référencer pour l'utiliser dans le code des activités de l'application.
- La classe R est final donc elle n'est pas héritable
- Toutes les ressources de l'applications sont utilisées dans le code à travers la classe statique R
  - Dès qu'elles sont déclarées dans le fichier XML
  - Le fichier associé est déposé dans le répertoire adéquat.

Éléments Graphiques de Base: Les Ressources : Classe R

- Les identifiants sont générés automatiquement dans la classe R au moment où on les crée.
- Quand, par exemple on a créé un champ de saisie (Editext) avec comme identifiant prenom, un attribut constant public static final int prenom est défini automatiquement dans la classe id et une valeur lui est attribuée. C'est un numéro unique qui identifie l'élément EditText que nous avons rajouté.
- Cet identifiant nous permettra de manipuler l'élément à partir du code java avec R.id.prenom.
- Les ressources sont utilisées de la manière suivante:

  android.R.type_ressource.nom_ressource

- Ils permettent de désigner les ressources dans le code Java ;
- Par exemple, le fichier activity_main.xml a comme identifiant

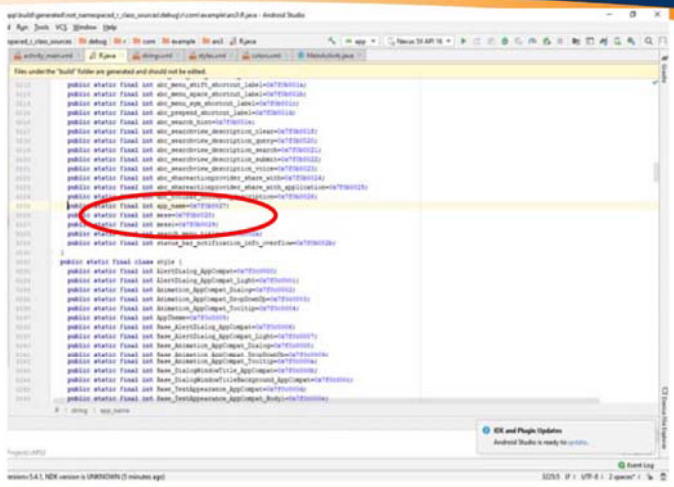
  R.layout.activity_main

Éléments Graphiques de Base: Les Ressources : Classe R

- Les identificateurs dans R.java font référence à des fichiers se trouvant dans le répertoire res.
  - les images sont accessibles par R.drawable.*nomImage* et correspondent à des fichiers images dans le répertoire res/drawable.
  - Les noms sont accessibles par R.string.*nom* et correspondent à des chaînes de caractères dans le fichier res/values/strings.xml. On récupère ces noms dans le code source par getResources().getString(R.string.*nom*);
  - Les composants graphiques ont un identifiant dans le fichier xml :
    - Cet identifiant est la valeur de l'attribut android:id = "@+id/IdentificateurAttribut". On récupère, dans le code, le composant graphique grâce à cet identifiant par l'appel findViewById(R.id. IditificateurAttribut);
    - Exemple : Accès à un composant bouton

      Button btn = findViewById(R.id.leId);

Éléments Graphiques de Base: Les Ressources : Classe R



Éléments Graphiques de Base: Les Ressources

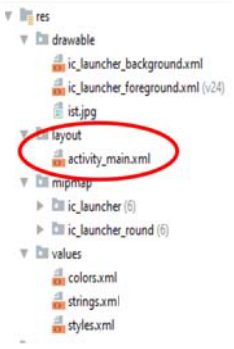
- [JAVA] La méthode setContentView peut prendre également l'identifiant d'un layout :

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

    }
}
```



Éléments Graphiques de Base: Les Ressources

- [JAVA] Une méthode spécifique pour les objets graphiques permet de les récupérer à partir de leur id, ce qui permet d'agir sur ces instances même si elles ont été créées via leur définition XML:

```
TextView txt = findViewById(R.id.texte1);
txt.setText("Hello FSM!");
```

Éléments Graphiques de Base: Les chaines

- Pour faciliter la traduction des applications, il est préférable d'utiliser des références à des chaînes de caractères dans les programmes ou les layouts :
- Les chaines constantes de l'application sont situées dans res/values/strings.xml.
- Ceci est pour réaliser l'internationalisation de l'application. Voici un exemple:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
        <string name="app_name">Application FSM</string>
        <string name="message">Bonjour mes Amis!</string>
</resources>
```

- [JAVA] La récupération de la chaine se fait via le code:

```
Resources res = getResources();
String hw = res.getString(R.string.message);
```

- [XML] La récupération de la chaine se fait via le code:

```
@string/message
```


Éléments Graphiques de Base: Les Ressources

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <TextView
        android:id="@+id/txt1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/message"
        android:textSize="40sp"
        android:gravity="center"
        android:textStyle="bold"
        android:textColor="@color/colorARS3"/>
</LinearLayout>
```



Éléments Graphiques de Base: Internationalisation

- Le système de ressources permet de gérer très facilement l'internationalisation d'une application.
- Il suffit de créer des répertoires values-XX où XX est le code de la langue que l'on souhaite implémenter.
- On place alors dans ce sous répertoire le fichier xml strings.xml contenant les chaines traduites associées aux mêmes clefs que dans values/strings.xml.
- On obtient par exemple pour les langues *es* et *fr* l'arborescence:

```
MyProject/res/
        values/strings.xml
        values-es/strings.xml
        values-fr/strings.xml
```

- Android chargera le fichier de ressources approprié en fonction de la langue du système.

Éléments Graphiques de Base: Autres Ressources

- D'autres ressources sont spécifiables dans res:
  - les menus (R.menu)
  - les images (R.drawable)
  - des couleurs (R.color):

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="colorPrimary">#3F51B5</color>
4      <color name="colorPrimaryDark">#303F9F</color>
5      <color name="colorAccent">#FF4081</color>
6  </resources>
7
```

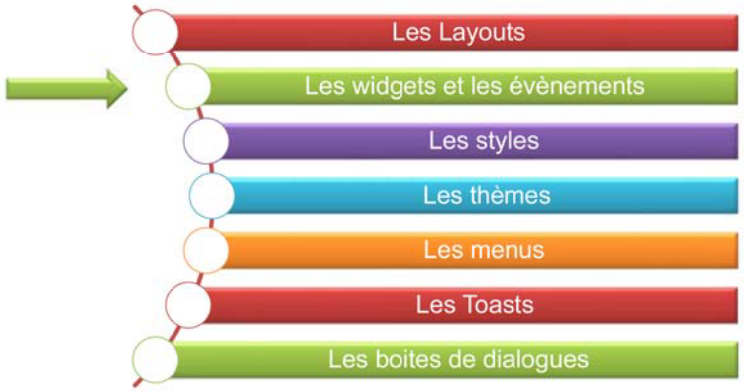
Éléments Graphiques de Base: Autres Ressources

- drawable-hdpi : contient toutes les images, bitmaps dont vous avez besoin pour votre application en haute résolution.
- drawable-ldpi : contient toutes les images, bitmaps dont vous avez besoin pour votre application en basse résolution.
- drawable-mdpi : contient toutes les images, bitmaps dont vous avez besoin pour votre application en moyenne résolution.
- Icon.png : l'icône de votre application, cette icône sera affichée sur le bureau.

Les Bonnes Pratiques

- Utiliser *wrap_content*, *mach_parent* ou les unités en *dp* quand on spécifie des dimensions dans un fichier de layoutXML
- Ne pas utiliser des valeurs en pixel
- Ne pas utiliser l'*AbsoluteLayout* (*deprecated*)
- Fournir des ressources alternative (pour les bitmaps notamment) qui correspondent à des densités d'écran différentes.

4. Interfaces graphiques



Utilisation des composants graphiques (widgets)

- Pour utiliser un composant graphique, il faut lui associer un id
  - Représenté dans le layout XML par l'attribut android:id
  - En général sous la forme: @+id/nom_id
    - @ : variable particulière, doit être parsée par le parseur XML
    - + : un identifiant avec la valeur nom_id sera généré, et ajouté automatiquement à la classe R
- Fichier R.java
  - Contient un ensemble de classes générées automatiquement à partir du répertoire res.
  - Chaque classe représente un ensemble de propriétés utilisables dans le code Java
  - Fichier généré, ne doit donc pas être modifié à la main

L'interface comme ressource

- Une interface graphique définie en XML sera aussi générée comme une ressource dans la classe statique R.
- Le nom du fichier xml, par exemple accueil.xml permet de retrouver le layout dans le code java à travers
  R.layout.accueil.
- Associer la première vue graphique à l'activité principale de l'application :

```
public void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.acceuil);
}
```


L'interface comme ressource

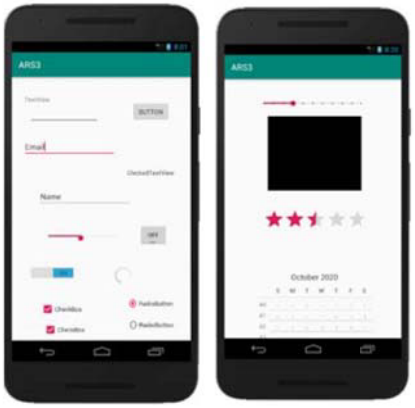
- Le layout reste modifiable à travers le code Java, comme tous les autres objets graphiques.
- Ainsi, on peut accéder à cet élément par son id et agir dessus a travers du code Java:

```
LinearLayout l = (LinearLayout)findViewById(R.id.accueilid);
l.setBackgroundColor(Color.BLACK);
```

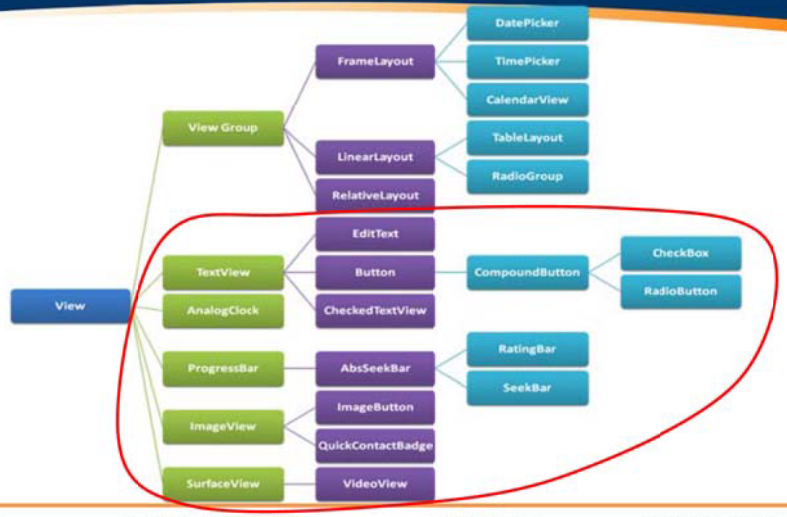
- Un Layout peut contenir des éléments graphiques, ou d'autres Layout

Les Composants graphiques

- Il existe plusieurs composants simples Android afin de construire des interfaces attirantes.
  - TextView : Label
  - EditText : Champ Text de Saisie
  - EditText : saisie préformatée (Email, Number, …)
  - CheckBox, RadioButton
  - Button : un bouton pour gérer des clicks
  - Calendrier
  - Progress bar, Rate bar, Seek bar
  - Toggle Button
  - ImageView, VideoView, MapView
  - …



Les Composants graphiques

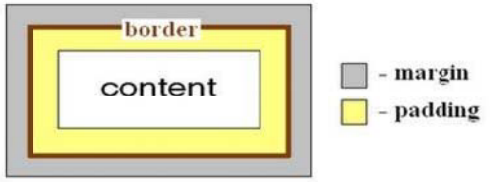


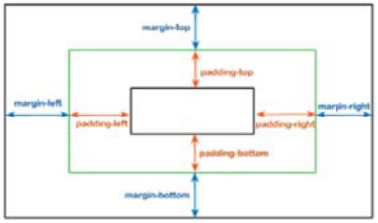
Les éléments graphiques de base : Attributs Communs

- **Identifiant : android:id="@+id/mon_ident"**
- **android:layout_width et android:layout_height:**
  - **="mach_parent"**: l'élément remplit tout l'élément parent
  - **="wrap_content"**: prend la place nécessaire à l'affichage
- **android:background: "@color/blue/#0000FF"**
- **android:visibility : visible / invisible**
- **android:layout_gravity :** est la gravité **extérieure** de la vue. Spécifie la direction dans laquelle la vue doit toucher la bordure de son parent. **left | center_horizontal | top | bottom | right**
- **android:gravity:** est la gravité **intérieure** de cette vue. Spécifie dans quelle direction son contenu doit s'aligner. **left | center_horizontal | top | bottom | right**

Les éléments graphiques de base : Attributs Communs

- **Android:layout_Padding : Espace entre le texte et les bords : `Padding, paddingRight, paddingLeft, paddingTop, paddingBottom`**
- **Android:layout_ Margin : Espace autour des bords : `layout_margin, layout_marginRight, layout_marginLeft, layout_marginTop, layout_marginBottom,`**





Les éléments graphiques de base : TextView

- **TextView : Les labels de texte, Sert de zone de texte non éditable**
  - **Propriétés importantes :**
    - **android:text : le texte du TextView**
    - **android:typeface : le type de police utilisée (monospace...)**
    - **android:textStyle : le style (italic pour l'italique, bold pour gras et bold|italic...)**
    - **android:textColor pour la couleur d'affichage du texte.**
    - **android:textSize pour la taille du texte exprimée en « sp ».**
  - Les valeurs des couleurs sont en hexadécimale en unités RGB (#RedRedGreenGreenBlueBlue : #FF0000 → rouge)

Les éléments graphiques de base : TextView

- **En XML:**

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/mess"
    android:textSize="25sp"
    android:textColor="#990000"/>
```

- Par la programmation **[JAVA]**

```
LinearLayout cadre = new LinearLayout(this);
cadre.setGravity(Gravity.CENTER);
cadre.setOrientation(LinearLayout.VERTICAL);
TextView txt = new TextView(this);
txt.setText("Test de Creation d'Interface");
cadre.addView(txt);
setContentView(cadre);
```

Les éléments graphiques de base : EditText

- EditText
  - Sert de zone de texte éditable
  - Propriétés importantes :
    - android:text : le texte du EditText
    - android:inputType : spécifie le type d'entrée attendu (numberDecimal, numberSigned, phone, date, textEmailAddress, textMultiLine, textPersonName)
    - android:hint : texte à afficher lorsque le composant est vide



Les éléments graphiques de base : EditText

- EditText
  - Par XML

```
<EditText
    android:id="@+id/email_address"
    android:layout_width="mach_parent"
    android:layout_height="wrap_content"
    android:hint="@string/email_hint"
    android:inputType="textEmailAddress" />
```

  - Par la programmation [JAVA]

```
EditText edit = new EditText(this);
edit.setText("saisir votre nom ici");
cadre.addView(edit);
```


Les éléments graphiques de base: ImageView

- ImageView
  - Affiche une image sans interaction
  - Propriétés importantes :
    - ➔ android:src : l'id du drawable
    - ➔ android:scaleType : contrôle comment l'image va être modifiée en fonction de la taille de la vue

Les éléments graphiques de base: ImageView

- Les images
  - Par XML

```
<ImageView
    android:id="@+id/imageView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    app:srcCompat="@drawable/image" />
```

  - Par la programmation [JAVA]:

```
ImageView image = new ImageView(this);
image.setImageResource(R.drawable.image);
cadre.addView(image);
```

Les éléments graphiques de base: Button

- **Button**
  - Un bouton qui renvoie un évènement lors de l'appui
  - Peut contenir un texte, une image ou les deux
  - Propriétés importantes :
    - android:text : le texte du bouton
    - android:onClick : abonne un listener sur un évènement clic de l'utilisateur
    - android:drawableLeft : une image à afficher avec les textes à la position souhaitée (Top, Bottom, Left, Right)
    - android:enable : actif ou inactif



Les éléments graphiques de base: Button

- **Les boutons :**
  - En XML:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/mess"/>
```

  - Par la programmation [**JAVA**]:

```
Button bouton = new Button(this);
bouton.setText("saisir votre nom ici");
cadre.addView(bouton);
```


Les éléments graphiques de base: Button

```
<ImageButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:scaleX="2"
    android:scaleY="2"
    android:src="@drawable/ic_person">
</ImageButton>
```



```
<Button
    android:id="@+id/btnV1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:drawableLeft="@drawable/ic_person"
    android:gravity="center"
    android:text="valider" />
```



Les éléments graphiques de base: Button : Gestion des Evènements

- La gestion des événements de *click* est réalisée par l'intermédiaire d'un listener
- Il existe trois manières pour gérer le click Button (définition du comportement du bouton) :
  - 1 ère Façon : on associe un call-back sur l'évènement CLICK
  - 2 éme Façon : A travers l'`interface OnClickListener` : En choisissant d'implémenter cette interface l'Activity courante s'engage à définir le contenu de la méthode OnClick
  - 3 éme Façon : A travers les attributs événementiels en XML

Les éléments graphiques de base: Button : Gestion des Evènements

- 1ere Façon :

1. Définir un objet Java de type android.widget.Button

```
private Button btn;
```

2. Initialiser cet objet en l'associant à son équivalent dans le fichier XML

```
btn = findViewById(R.id.button1);
```

3. Définir le comportement de ce bouton

```
btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

    }
});
```


Les éléments graphiques de base: Button : Gestion des Evènements

- 2 éme Façon :

1. Définir un objet Java de type android.widget.Button
2. Initialiser cet objet en l'associant à son équivalent dans le fichier XML
3. Implémenter l'interface View.OnClickListener

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener
```

4. Surcharger la méthode onClick

```
@Override
public void onClick(View v) {
    if (v.getId()==R.id.button){
        // comportement du bouton
    }
}
```

cette méthode sera commune à tous les éléments cliquables, il faut donc distinguer le comportement selon l'identifiant de l'élément cliqué

5. Définir la classe en cours comme étant le Listener du bouton myBytton

```
btn.setOnClickListener(this);
```


Les éléments graphiques de base: Button : Gestion des Evènements

- 3 éme Façon :

1. Dans le fichier layout XML, associer au bouton le nom d'une méthode qui définira son comportement au clic

```
android:onClick="onClickHello"
```

2. Dans la classe Java de l'activité, créer et implémenter cette méthode en respectant les contraintes suivantes:
   - Elle doit retourner void
   - Elle doit porter le nom défini dans le fichier layout XML
   - Elle doit définir un paramètre de type android.view.View

```
public void onClickHello(View v){
    // comportement du bouton
}
```


Les éléments graphiques de base: Button : Gestion des Evènements

- Liste des interfaces de listeners graphiques

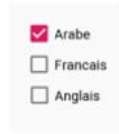
OnClickListener	// clic
OnLongClickListener	// clic long
OnDragListener	// glissement
OnTouchListener	// touché
OnHoverListener	// survol
OnKeyListener	// frappe de clavier
OnAttachStateChangeListener	// changement de l'attachement
OnLayoutChangeListener	// changement du layout
OnCreateContexMenuListener	// creation du menu contextuel
OnFocusChangeListener	// changement du focus
OnGenericMotionListener	// un mouvement (mouse, pen, finger, …)
OnSystemUiVisibilityChangeListener	// changement de la visibilité de la barre d'état

Exercice



Les éléments graphiques de base: CheckBox

- **Checkbox**
  - Une case à cocher
  - Propriétés importantes :
    - android:text : le texte de la case
    - android:checked : spécifie si le composant est coché par défaut
  - **XML**




```
<CheckBox
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Arabe"
    android:id="@+id/chkAr"
    android:checked="true" />
```


Les éléments graphiques de base: CheckBox

- Java

```
chk1.setOnClickListener(new View.OnClickListener() {
   public void onClick(View v) {
        if (chk1.isChecked())
            ////////////////
        else
            ////////////////
    }    });
```

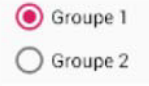
Ou bien

```
btn.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        if (chk1.isChecked())
          ////////////////
        else
          ////////////////
        if(chk2.isChecked())
          ////////////////
        else
        ////////////////
 Toast.makeText(MainActivity.this, "..............", Toast.LENGTH_SHORT).show();} });
```


Les éléments graphiques de base: RadioButton

- **RadioButton**
  - Un Bouton d'Option
  - Propriétés importantes :
    - android:text : le texte de l'option
    - android:checked : spécifie si le composant est coché par défaut

```
<RadioButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Groupe 1"
    android:id="@+id/idG1"/>
<RadioButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Groupe 2"
    android:id="@+id/idG2"/>
```



Les éléments graphiques de base: RadioButton

- **RadioGroup et RadioButton**
  - Un groupe de boutons dépendants
  - Pour que les RadioButton soient dépendants, il faut qu'ils soient contenus dans le même groupe

```
<RadioGroup
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
  <RadioButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Groupe 1"
    android:id="@+id/idG1"/>
  <RadioButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Groupe 2"
    android:id="@+id/idG2"/>
</RadioGroup>
```


Les éléments graphiques de base: RadioButton

- **RadioButton et évènements**
  - Deux façons :
    - Créer un listener par RadioButton
    - Utiliser la même méthode pour tous les RadioButton d'un même RadioGroup. Dans la méthode on sait quel RadioButton appelle grâce à l'attribut View

Les éléments graphiques de base: RadioButton

- Créer un listener par RadioButton

```
rd1 = findViewById(R.id.idRD1);

rd1.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        ////////////////////
    }
});
```


Les éléments graphiques de base: RadioButton

- Utiliser la même méthode pour tous les RadioButton d'un même RadioGroup.

```
<RadioButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="option1"
    android:id="@+id/idOp1"
    android:onClick="validerRadio"/>
<RadioButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="option2"
    android:id="@+id/idOp2"
    android:onClick="validerRadio"/>
```

```
public void validerRadio(View v){
 switch (v.getId()){
     case R.id.idOp1: ////////;
     case R.id.idOp2: ////////;
     default: }
}
```